

CUB3D — GUIA DE SPRINTS

Documentação Técnica de Implementação • Exclusivo em Preto e Branco (1 Sprint por Página)

SPRINT 3 • PERÍODO: 29/07 → 02/08

ETAPA 03 — Setup da MLX42

Dificuldade:

★★★★☆☆

Próxima Etapa:

04_player.md

OBJETIVO

Inicializar completamente a **MLX42**, criando toda a infraestrutura gráfica necessária para o restante do projeto. Ao final desta etapa o projeto deverá inicializar a MLX42, criar uma janela, criar o framebuffer principal, registrar os hooks, renderizar teto e chão e fechar corretamente a aplicação. **(Nenhum raycasting será implementado nesta etapa).**

RESULTADO ESPERADO

```
./cub3D maps/tests/valid_basic.cub
-> MLX inicializada
-> Janela criada
-> Framebuffer criado
-> Hooks registrados
-> Tela exibe teto e chão
```

ARQUIVOS ENVOLVIDOS

```
src/game/: init_game.c, load_textures.c
src/rendering/: put_pixel.c, render_background.c,
render_frame.c
src/hooks/: close_hook.c
```

FLUXO GERAL

```
main() -> mlx_init() -> janela -> imagem -> hooks -> fundo -> mlx_loop()
```

ORDEM DE IMPLEMENTAÇÃO

- Parte 1 — Inicialização MLX & Parte 2 — Janela
- Criar contexto com `mlx_init()`. Criar janela via `mlx_new_window()` (`WIDTH`, `HEIGHT`, `"cub3D"`).
- Parte 3 — Framebuffer & Parte 4 — Inserção da Imagem
- Criar framebuffer em `game->screen` via `mlx_new_image()`. Adicionar à janela com `mlx_image_to_window()`.
- Parte 5 — `put_pixel.c` & Parte 6 — Renderização do Fundo
- Criar `put_pixel()` com validações de limite. Renderizar teto e chão com `map.ceiling_color` e `map.floor_color`.
- Parte 7 — `render_frame.c`, Parte 8 — Hooks & Parte 9 — Loop
- Centralizar chamadas em `render_frame()`. Tratar `ESC` e fecho de janela em `close_hook.c`. Rodar `mlx_loop()`.

CHECKLIST & TESTES

- [] MLX inicializada & Janela criada
- [] Framebuffer criado e inserido na janela
- [] `put_pixel()`, `render_background()` OK
- [] `ESC` e botão fechar funcionando
- [] Norminette & Valgrind aprovados

CASOS CRÍTICOS & DOD

Atenção: Se MLX falhar, emitir mensagem e liberar memória. Fechamento deve ser limpo sem leaks.

DoD: Fundo renderizado, parser integrado, 0 leaks/segfaults, pronto para Player.

CUB3D — GUIA DE SPRINTS

Documentação Técnica de Implementação • Exclusivo em Preto e Branco (1 Sprint por Página)

SPRINT 3 • PERÍODO: 02/08 → 05/08

ETAPA 04 — Player e Controles

Dificuldade:

★★★★☆

Próxima Etapa:

05_raycasting.md

OBJETIVO

Implementar a lógica do jogador: inicializar pelo parser, mover via **WASD**, rotacionar via ← →, tratar colisões contra paredes e atualizar câmera em tempo real. **(Sem raycasting ainda).**

ARQUIVOS ENVOLVIDOS

src/game/: `init_player.c`
src/hooks/: `key_hook.c`, `movement.c`, `rotation.c`, `collision.c`, `update_hook.c`
src/utlis/: `math_utils.c`

ESTRUTURA T_PLAYER

```
typedef struct s_player {
    double pos_x, pos_y;
    double dir_x, dir_y;
    double plane_x, plane_y;
    double move_speed, rot_speed;
} t_player;
```

ORDEM DE IMPLEMENTAÇÃO

- Parte 1 — Inicialização (init_player.c)**
Converter posição do parser em coordenadas. Ajustar vetores de direção/plano:
 - **N:** dir = (0, -1) | **S:** dir = (0, 1) | **E:** dir = (1, 0) | **W:** dir = (-1, 0)
- Parte 2 — Key Hook & Parte 3 — Movimento (movement.c)**
Capturar `W, A, S, D, ←, →`. Criar `move_forward()`, `move_backward()`, `move_left()`, `move_right()`.
- Parte 4 — Rotação (rotation.c)**
Rotacionar vetores de direção e plano via `rotate_left()` e `rotate_right()` (360° sem perda de precisão).
- Parte 5 — Colisão (collision.c) & Parte 6 — Update Hook**
Bloquear passagem em células com `'1'` (testar cantos, paredes e corredores). Integrar ciclo em `update_hook.c`.

CHECKLIST & TESTES

- [] Player inicializado na posição/direção correta
- [] Movimentação WASD suave e contínua
- [] Colisão impede atravessar paredes/cantos
- [] Norminette & Valgrind sem erros

CASOS CRÍTICOS & DOD

Atenção: Evitar acúmulo de erro em rotações prolongadas. Permitir deslizar pelas paredes.

DoD: Player movendo, rotacionando e colidindo perfeitamente, pronto para Raycasting.

CUB3D — GUIA DE SPRINTS

Documentação Técnica de Implementação • Exclusivo em Preto e Branco (1 Sprint por Página)

SPRINT 4 • PERÍODO: 05/08 → 08/08

ETAPA 05 — Raycasting

Dificuldade:

★★★★★

Próxima Etapa:

06_texture_mapping.md

OBJETIVO

Implementar o motor 3D de raycasting: lançar um raio por coluna da tela, executar o algoritmo DDA, calcular a distância perpendicular (sem efeito fisheye), determinar a altura da parede e desenhar colunas sólidas.

ARQUIVOS ENVOLVIDOS

src/raycasting/: ray_init.c, ray_distance.c, ray_dda.c, raycast.c

src/rendering/: draw_line.c, render_frame.c

ESTRUTURA T_RAY

```
typedef struct s_ray {
    double camera_x, ray_dir_x, ray_dir_y;
    int map_x, map_y;
    double side_dist_x, side_dist_y;
    double delta_dist_x, delta_dist_y;
    double perp_wall_dist;
    int step_x, step_y, side;
    int line_height, draw_start, draw_end;
} t_ray;
```

ORDEM DE IMPLEMENTAÇÃO

- Parte 1 — Raio & Parte 2 — Delta Distance
- Calcular camera_x ([-1, +1]), ray_dir e map_x/y. Calcular delta_dist_x/y em ray_distance.c.
- Parte 3 — Step & Parte 4 — Side Distance
- Definir step_x/y (+1 ou -1) e calcular a primeira interseção com as linhas do grid em side_dist_x/y.
- Parte 5 — Algoritmo DDA (ray_dda.c)
- Avançar o raio no grid até encontrar parede (perform_dda()). Armazenar a face atingida em side.
- Parte 6 — Distância Perpendicular & Parte 7 — Altura da Parede
- Calcular perp_wall_dist. Obter line_height, draw_start e draw_end para centralizar a projeção.
- Parte 8 — Renderizar Coluna & Parte 9 — Loop (raycast.c)
- Desenhar linha vertical em draw_line.c. Iterar x de 0 até WIDTH executando todos os raios.

CHECKLIST & TESTES

- [] Cálculo de camera_x e delta_dist corretos
- [] DDA funcional sem loops infinitos
- [] Perp_wall_dist sem efeito fisheye
- [] Norminette & Valgrind sem alertas

CASOS CRÍTICOS & DOD

Atenção: Prevenir divisão por zero para raios paralelos aos eixos. Utilizar tipo double em toda a matemática.

DoD: Projeção 3D sólida sem texturas funcionando perfeitamente, pronta para Mapeamento de Texturas.

CUB3D — GUIA DE SPRINTS

Documentação Técnica de Implementação • Exclusivo em Preto e Branco (1 Sprint por Página)

SPRINT 5 • PERÍODO: 12/08 → 15/08

ETAPA 06 — Texture Mapping

Dificuldade:

★★★★★

Próxima Etapa:

07_integration.md

OBJETIVO

Mapear e aplicar texturas às paredes 3D: carregar 4 texturas XPM (NO, SO, EA, WE), calcular ponto de impacto e coordenadas UV da textura e aplicar sombreamento em paredes laterais.

ARQUIVOS ENVOLVIDOS

src/game/: load_textures.c

src/rendering/: texture_sampling.c, draw_line.c

src/raycasting/: ray_texture.c

ESTRUTURA T_TEXTURE

```
typedef struct s_texture {
    mlx_texture_t *texture;
    uint32_t      *pixels;
    int            width;
    int            height;
} t_texture;
```

ORDEM DE IMPLEMENTAÇÃO

- Parte 1 — Carregar Texturas & Parte 2 — Seleção de Textura
- Carregar XPMs em load_textures.c. Selecionar textura (Norte, Sul, Leste, Oeste) via select_texture().
- Parte 3 — Posição na Parede (wall_x) & Parte 4 — Coordenada X (texture_x)
- Obter wall_x (impacto exato [0.0 - 1.0]). Converter em texture_x (corrigindo inversões de orientação).
- Parte 5 — Escala Vertical & Parte 6 — Amostragem
- Calcular o step de avanço na textura. Obter a cor do pixel em texture_y via sample_texture().
- Parte 7 — Desenhar Coluna & Parte 8 — Sombreamento
- Renderizar parede em draw_textured_wall(). Aplicar tom mais escuro em paredes horizontais em apply_shading().

CHECKLIST & TESTES

- [] 4 texturas carregadas sem memory leak
- [] Seleção de textura N, S, E, W perfeita
- [] Mapeamento UV (wall_x, texture_x, texture_y) sem distorção
- [] Sombreamento de profundidade funcional
- [] Norminette & Valgrind totalmente zerados

CASOS CRÍTICOS & DOD

Atenção: Proteger contra acessos fora da largura/altura do buffer de pixels. Corrigir texturas espelhadas.

DoD: Paredes texturizadas estilo Wolfenstein 3D completas e sem leaks, pronto para Integração Final.